ACTIVIDAD 3: CONCEPTUALIZACIÓN BÁSICA

PROYECTO «MI LENGUAJE, MI COMPILADOR»

DISEÑO DEL LENGUAJE DeLuX

REALIZADO POR:

DEINER DAVID BELLO GONZÁLEZ

LUVIS YORFARI CORREA MOSQUERA

DOCENTE

HASLER GAMBOA BERRIO

ASIGNATURA:

COMPILADORES

FUNDACIÓN UNIVERSITARIA CLARETIANA

PROGRAMA DE INGENIERÍA DE SISTEMAS

CURSO: COMPILADORES

2026

CONTENIDO

Introducción.....3

1. Descripción del lenguaje DeLuX.....4

1.1 Tipos de datos.....4

1.2 Instrucciones básicas.....4

1.3 Estructuras de control.....5

1.4 Operadores.....5

1.5 Reglas para escribir en DeLuX.....5

2. Fases del compilador.....7

2.1 Diagrama de las fases.....7

2.2 Qué hace cada fase en DeLuX.....8

3. Programas correctos.....9

4. Programas con error y su análisis.....10

4.1 Programa con error léxico.....10

4.2 Programa con error sintáctico.....10

4.3 Programa con error semántico.....11

4.4 Tabla resumen de errores.....12

5. Decisiones de diseño.....13

Glosario.....14

Referencias.....15

Introducción

En esta actividad el grupo diseña el lenguaje que construirá durante el proyecto «Mi lenguaje, mi compilador». El lenguaje se llama **DeLuX** y es un lenguaje de **propósito general**: no está hecho para un solo problema, sino para escribir programas de cualquier tipo, como hacer cálculos, pedir datos al usuario, tomar decisiones y repetir tareas.

La idea es parecida a Python, que sirve para inteligencia artificial, páginas web o análisis de datos sin estar hecho solo para una de esas cosas. DeLuX usa palabras clave en español y, por indicación del docente, no incluye funciones, para mantener el compilador en un tamaño manejable.

El documento describe los tipos de datos, las instrucciones y su forma de escritura, muestra las fases del compilador aplicadas a DeLuX, presenta tres programas correctos y tres con un error intencional (léxico, sintáctico y semántico), y explica en qué fase se detecta cada error y por qué. Los archivos de DeLuX usan la extensión `.dlx`.

1. Descripción del lenguaje DeLuX

DeLuX cumple los mínimos del proyecto: instrucciones propias, variables con números, repetición, condición, suma, resta, multiplicación y las comparaciones >, < y ==. Además agrega lo necesario para que sea útil en general: texto, entrada de datos, división y operadores lógicos. Cada línea contiene una sola instrucción y los bloques se cierran con `fin`.

1.1 Tipos de datos

Tabla 1

Tipos de datos de DeLuX

Tipo	Cómo se escribe	Ejemplo
Número	Entero o decimal; el decimal usa punto.	`edad = 20`, `nota = 4.5`
Texto	Entre comillas dobles, en una sola línea.	`nombre = "Ana"`
Lógico	Las palabras `verdadero` o `falso`.	`activo = verdadero`

Nota. las variables no se declaran con tipo: toman el tipo del primer valor que reciben, como en Python.

1.2 Instrucciones básicas

Tabla 2

Instrucciones básicas de DeLuX

Instrucción	Cómo se escribe	Qué hace	Ejemplo
Asignación	`variable = expresión`	Guarda un valor con un nombre. Si la variable ya existe, cambia su valor.	`total = precio * 2`
mostrar	`mostrar expresión`	Muestra en pantalla un texto, un número o el resultado de una operación.	`mostrar "Hola"`
leer	`leer variable`	Pide un valor al usuario y lo guarda en la variable.	`leer edad`

1.3 Estructuras de control

Tabla 3

Condición, repeticiones y comentarios

Estructura	Cómo se escribe	Qué hace
Condición	`si condición entonces` … `sino` … `fin`	Si la condición se cumple, ejecuta el primer bloque; si no, el segundo. El `sino` es opcional.
Repetición fija	`repetir N veces` … `fin`	Ejecuta N veces las líneas del bloque. N puede ser un número o una variable.
Repetición condicional	`mientras condición hacer` … `fin`	Repite el bloque mientras la condición sea verdadera.
Comentario	`# texto`	El compilador ignora todo lo que sigue al # hasta el final de la línea.

1.4 Operadores

Tabla 4

Operadores disponibles en DeLuX

Tipo	Símbolos	Ejemplo
Aritméticos	`+ - * /`	`promedio = suma / 3`
Comparación	`> < == >= <= !=`	`si edad >= 18 entonces`
Lógicos	`y o no`	`si nota > 3 y asistencia > 80 entonces`
Asignación	`=`	`contador = 0`
Agrupación	`( )`	`resultado = (a + b) * 2`

Nota. el + también une textos: `mostrar "Hola " + nombre`.

1.5 Reglas para escribir en DeLuX

- **Palabras reservadas:** `mostrar`, `leer`, `si`, `entonces`, `sino`, `fin`, `repetir`, `veces`, `mientras`, `hacer`, `y`, `o`, `no`, `verdadero` y `falso`. No se pueden usar como nombre de variable.

- **Variables:** empiezan con una letra y siguen con letras, números o guion bajo. Ejemplos: `total`, `nota_final`, `x2`.
- **Números:** enteros (`15`) o decimales con punto (`3.5`). No llevan separador de miles.
- **Texto:** va entre comillas dobles y debe cerrarse en la misma línea.
- **Símbolos no permitidos:** cualquier carácter que no aparezca en estas reglas, como `@`, `$`, `;` o `&`, produce un error léxico.

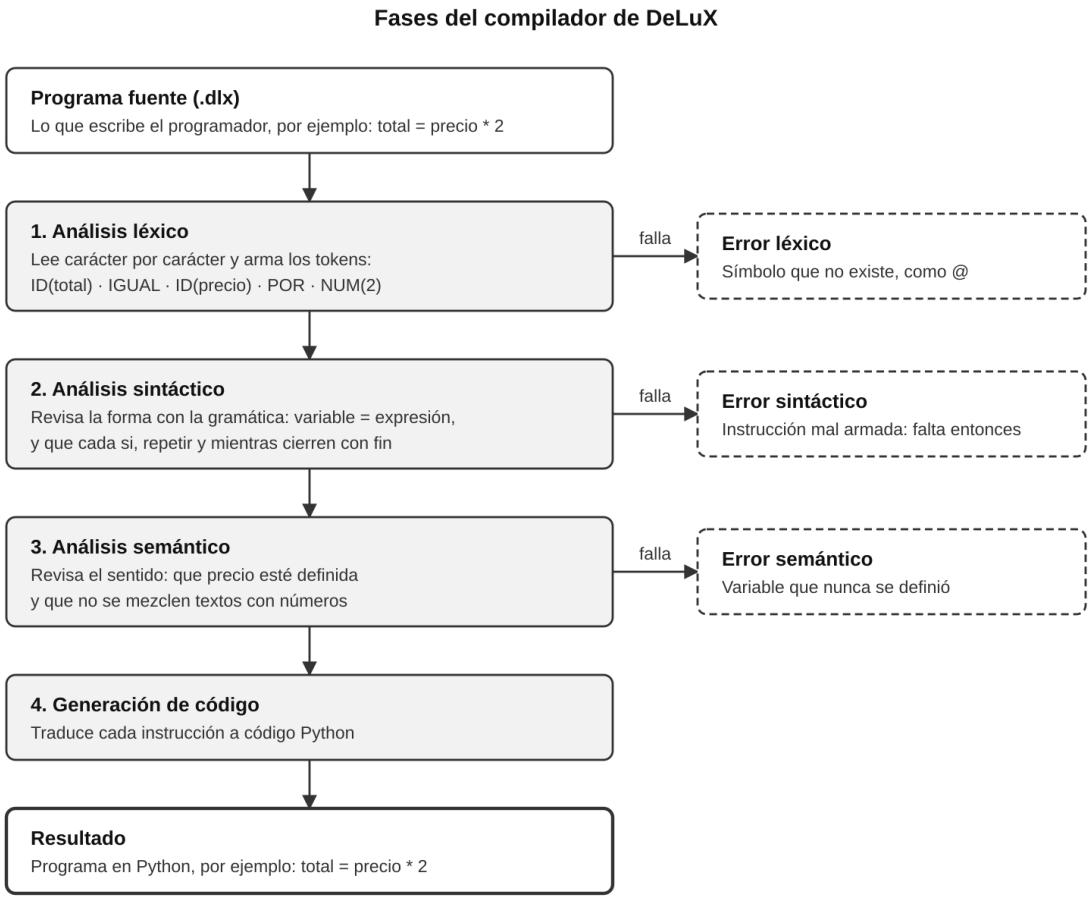
2. Fases del compilador

2.1 Diagrama de las fases

La Figura 1 muestra el recorrido de un programa DeLuX por las cuatro fases. Si una fase encuentra un error, el compilador se detiene y muestra el mensaje con la línea y la columna donde ocurrió.

Figura 1

Fases del compilador de DeLuX y sus salidas de error



Nota. elaboración propia. La instrucción `total = precio * 2` se sigue a través de cada fase.

2.2 Qué hace cada fase en DeLuX

Análisis léxico

Separa el texto en tokens (palabras reservadas, variables, números, textos y símbolos) y avisa si aparece un carácter que DeLuX no conoce, como `@`. Por ejemplo, `total = precio * 2` se convierte en cinco tokens: ID(total), IGUAL, ID(precio), POR y NUM(2).

Análisis sintáctico

Revisa con la gramática que cada instrucción esté bien armada: que una asignación tenga la forma `variable = expresión`, que todo `si` lleve `entonces`, que todo `mientras` lleve `hacer` y que cada bloque cierre con `fin`.

Análisis semántico

Revisa que el programa tenga sentido. Usa la tabla de símbolos para comprobar que las variables existan antes de usarse, y además verifica que no se resten, multipliquen o dividan textos con números, y que `repetir` no reciba una cantidad negativa.

Generación de código

Traduce el programa ya revisado a Python, que luego se puede ejecutar. Por ejemplo, `mostrar "Hola"` se convierte en `print("Hola")` y `repetir 3 veces` se convierte en `for _ in range(3):`.

3. Programas correctos

Programa 1 — Área de un rectángulo (area.dlx)

Usa variables, multiplicación y la instrucción `mostrar` con texto y con números.

```
# Calcula el area de un rectangulo
base = 8
altura = 5
area = base * altura
mostrar "El area es:"
mostrar area
```

Programa 2 — Tabla de multiplicar (tabla.dlx)

Pide un número al usuario y muestra su tabla del 1 al 10. Usa `leer`, `repetir`, suma y multiplicación.

```
leer numero
contador = 1
repetir 10 veces
    mostrar numero * contador
    contador = contador + 1
fin
```

Programa 3 — Nota y cuenta regresiva (nota.dlx)

Decide si una nota aprueba y luego hace una cuenta regresiva. Usa condición, comparación, resta y `mientras`.

```
leer nota
si nota >= 3.0 entonces
    mostrar "Aprobado"
sino
    mostrar "Reprobado"
fin
intentos = 3
mientras intentos > 0 hacer
    mostrar intentos
    intentos = intentos - 1
fin
```

4. Programas con error y su análisis

Cada programa tiene un solo error, puesto a propósito. Cada fase solo revisa lo que le corresponde, por eso el error lo detecta la primera fase capaz de notarlo.

4.1 Programa con error léxico

Programa 4 — error_lexico.dlx

```
precio = 2500
cantidad = 3
total = precio * cantidad @
mostrar total
```

¿Cuál es el error?

En la línea 3, columna 27, aparece el carácter `@`, que no pertenece al lenguaje. Mensaje esperado: «Línea 3, columna 27: carácter no reconocido '@'».

¿En qué fase se detecta y por qué?

Se detecta en el **análisis léxico**. Esta fase lee carácter por carácter y arma los tokens. Al llegar a `@` no puede formar ningún token válido, porque no es letra, ni número, ni comilla, ni un símbolo de DeLuX. Por eso falla aquí, antes de que el analizador sintáctico revise si la instrucción está bien armada.

4.2 Programa con error sintáctico

Programa 5 — error_sintactico.dlx

```
leer edad
si edad >= 18
    mostrar "Mayor de edad"
fin
```

¿Cuál es el error?

En la línea 2 la condición `si` no tiene la palabra `entonces`. Mensaje esperado: «Línea 2, columna 14: se esperaba 'entonces' después de la condición».

¿En qué fase se detecta y por qué?

Se detecta en el **análisis sintáctico**. Todos los tokens existen, así que el léxico no encuentra problema. El error está en el orden: la gramática dice que una condición se escribe `si condición entonces`. Después de leer `18`, el analizador espera `entonces`, pero encuentra el salto de línea.

4.3 Programa con error semántico

Programa 6 — error_semantico.dlx

```
suma = 0
repetir 5 veces
    suma = suma + valor
fin
mostrar suma
```

¿Cuál es el error?

En la línea 3, columna 19, se usa la variable `valor`, que nunca se definió. Mensaje esperado: «Línea 3, columna 19: la variable ‘valor’ no está definida».

¿En qué fase se detecta y por qué?

Se detecta en el **análisis semántico**. `valor` es un nombre válido, así que pasa el léxico, y `suma = suma + valor` respeta la forma `variable = expresión`, así que pasa el sintáctico. Pero no tiene sentido: el programa no sabe qué número sumar. Solo el semántico lo detecta, porque es la fase que consulta la tabla de símbolos con las variables definidas.

4.4 Tabla resumen de errores

Tabla 5

Errores intencionales, fase que los detecta y mensaje propuesto

Programa	Línea	Descripción del problema	Fase que lo detecta	Mensaje de error propuesto
Programa 4	3	El carácter «@» no es un símbolo válido de DeLuX.	Análisis léxico	Línea 3, columna 27: carácter no reconocido ‘@’.
Programa 5	2	La condición «si» no tiene la palabra «entonces».	Análisis sintáctico	Línea 2, columna 14: se esperaba ‘entonces’ después de la condición.
Programa 6	3	Se usa la variable «valor», que no fue definida.	Análisis semántico	Línea 3, columna 19: la variable ‘valor’ no está definida.

5. Decisiones de diseño

La primera versión del lenguaje estaba pensada solo para controlar un semáforo. Tras la revisión del docente, se cambió a un lenguaje de propósito general, porque el objetivo es un lenguaje que sirva para escribir cualquier programa, como ocurre con Python.

Se eligieron palabras clave en español (`mostrar`, `leer`, `si`, `mientras`) para que los programas se lean casi como instrucciones normales. Los bloques se cierran con `fin` en lugar de depender de la sangría, como hace Python, porque así la estructura es más fácil de revisar en el análisis sintáctico.

No se incluyeron funciones, por indicación del docente, para que el compilador no se vuelva demasiado complejo. Como resultado del compilador se eligió la traducción a Python, una de las opciones de la Actividad 7, porque permite ejecutar los programas de DeLuX sin construir una máquina propia.

Para la Actividad 7 quedan definidas tres reglas semánticas: no usar variables sin definir, no hacer operaciones aritméticas entre texto y números (salvo unir textos con +), y no aceptar una cantidad negativa en `repetir`.

Glosario

Lenguaje de propósito general: lenguaje que sirve para escribir programas de muchos tipos, no solo para un problema específico.

Token: unidad mínima con significado que reconoce el analizador léxico, como una palabra reservada, un número o un símbolo.

Palabra reservada: palabra que el lenguaje usa para sus instrucciones y que no puede usarse como nombre de variable.

Gramática: conjunto de reglas que indica cómo deben combinarse los tokens para formar instrucciones válidas.

Tabla de símbolos: estructura donde el compilador guarda las variables definidas y su tipo.

Error léxico: aparición de un carácter que no pertenece al lenguaje.

Error sintáctico: instrucción con tokens válidos pero mal ordenados o incompletos.

Error semántico: programa bien escrito que no tiene sentido, como usar una variable que no existe.

Generación de código: fase que traduce el programa revisado a otra forma ejecutable; en DeLuX, a Python.

Referencias

Fuente de la actividad:

Gamboa Berrio, H. (2026). *Proyecto de compiladores: Mi lenguaje, mi compilador* [Guía de actividad]. Fundación Universitaria Claretiana.

Marco teórico de compiladores:

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson / Addison-Wesley.

Wirth, N. (1996). *Compiler construction*. Addison-Wesley.

Herramientas de asistencia:

Anthropic. (2026). *Claude* [Asistente de inteligencia artificial]. Utilizado para organizar el documento y revisar la redacción de las explicaciones.